COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CpE212	**SECTION:**	CpE31s2
DATE PERFORMED:	Oct 28, 2026	**DATE SUBMITTED:**	Oct 29, 2026
NAME:	Jeus Guille V. Pizarro	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 12 **Build a Sample Web App in a Docker Container**

1. PROCEDURE OUTPUT

What procedures did you perform?

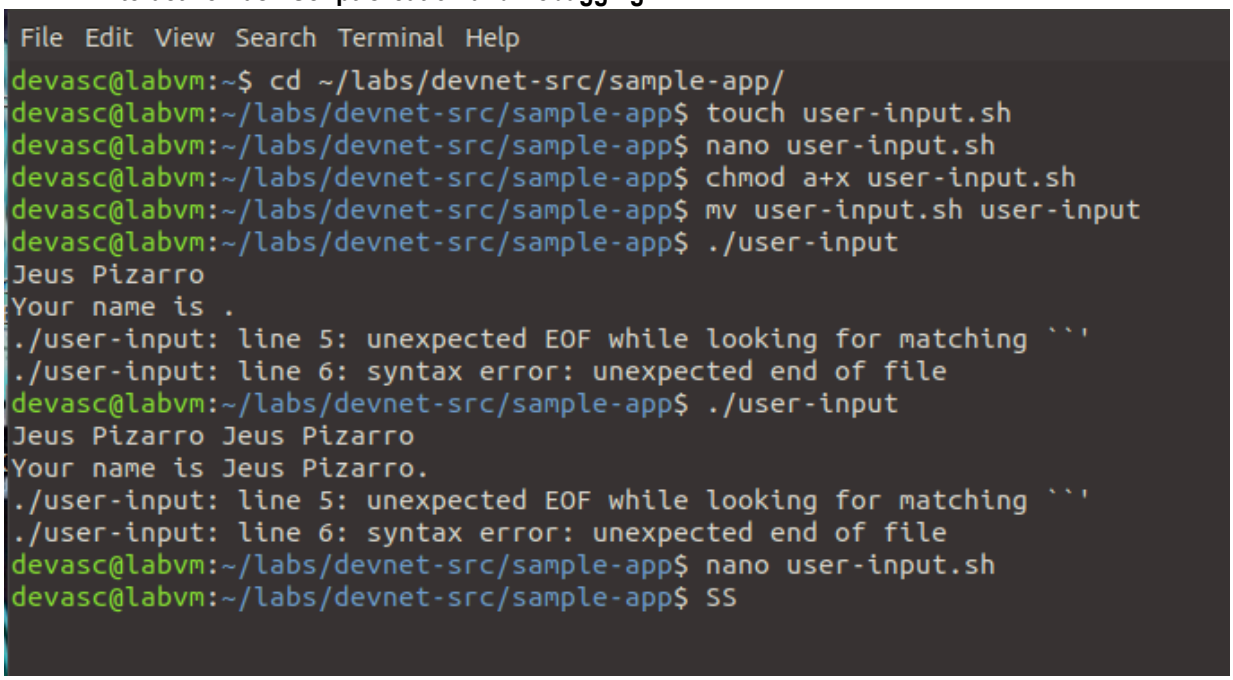
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PART 1: DEVASC-LABVM Environment Setup



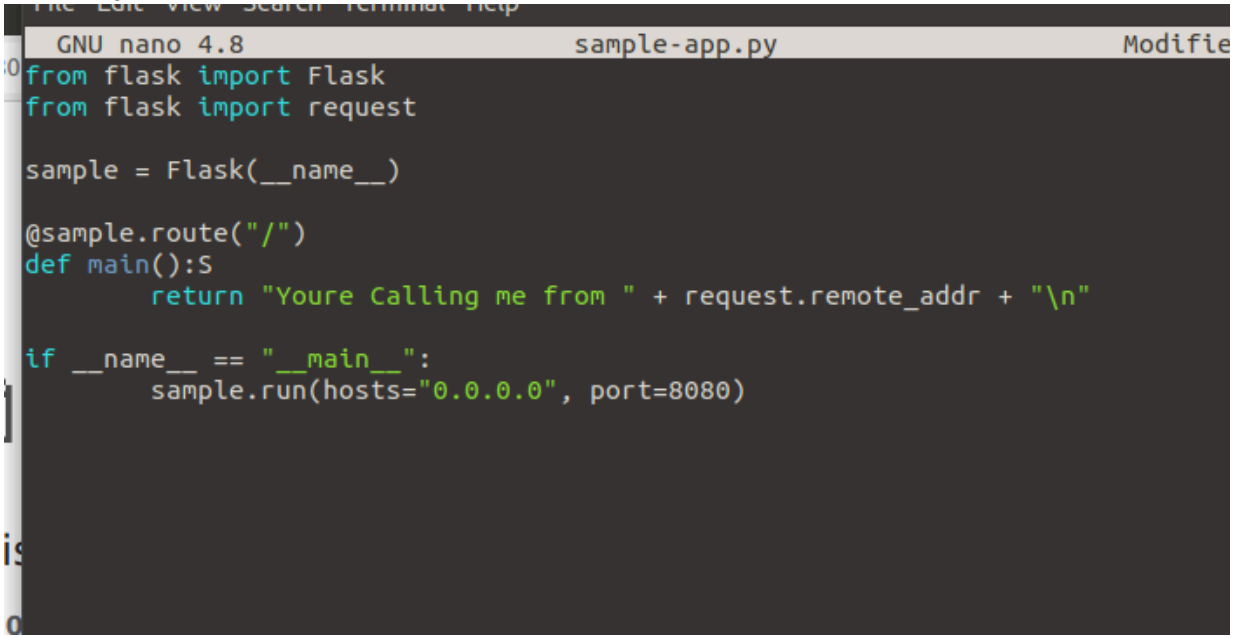
The output demonstrates the successful initialization of the DEVASC-LABVM virtual machine in Oracle VirtualBox. Shortcuts for essential development tools—including VS Code, Terminal, Packet Tracer, Chromium, Postman, draw.io, and lab repositories—are present and operational on the desktop.

PART 2: Interactive Bash Script Creation and Debugging

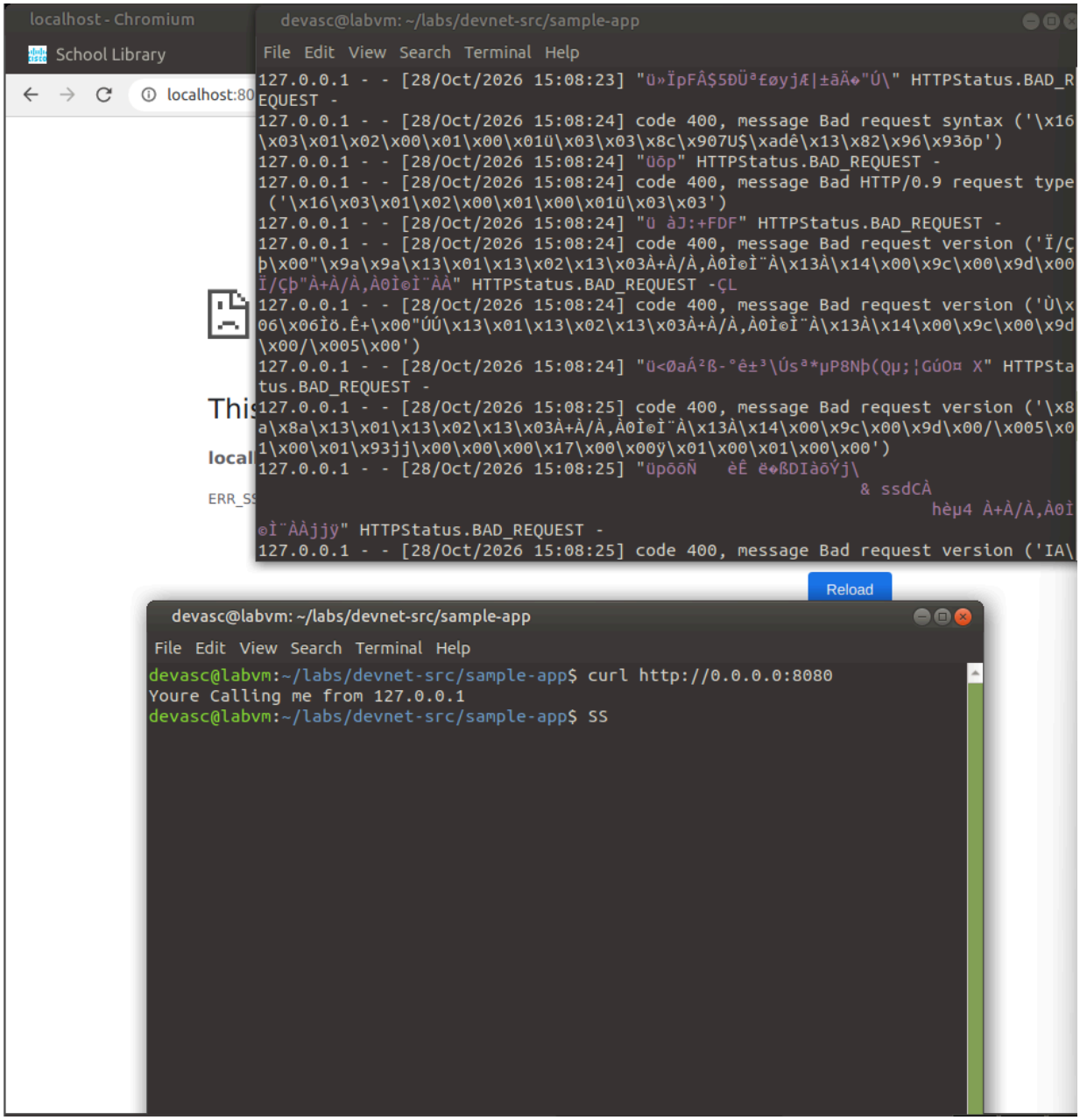


The terminal session records the creation and execution of an interactive shell script in *~/labs/devnet-src/sample-app/*. After fixing an initial quote syntax error using nano, the script executed properly to prompt for user input and print the full name.

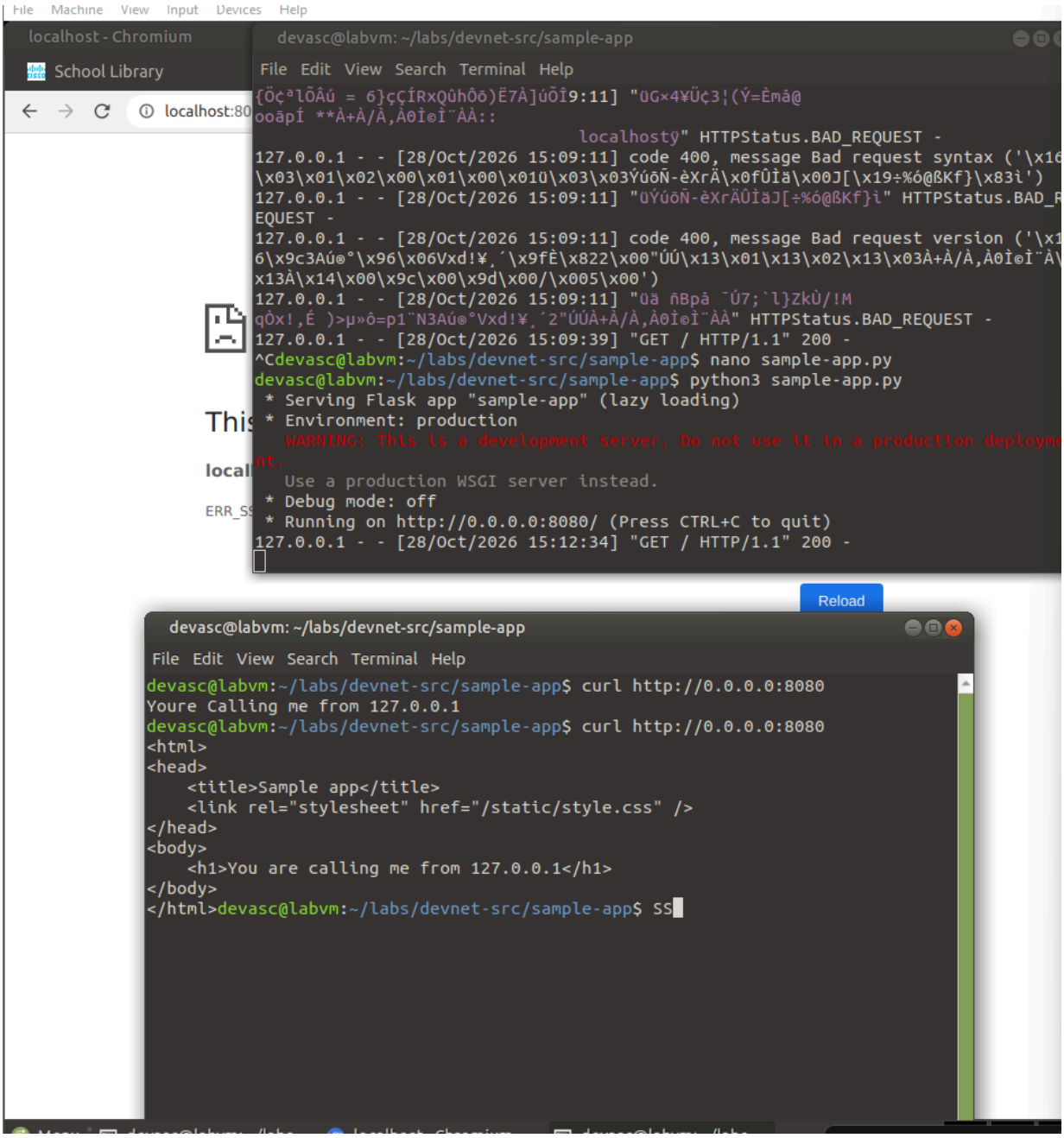
PART 3: Python Flask Web Application Code and Initial Test



The code listing defines a minimal Python Flask web application in *sample-app.py* that serves requests on port 8080 and returns the client's IP address. Terminal testing via cURL verifies successful HTTP responses showing local connection from 127.0.0.1.



PART 4: Web Application Server Execution and HTML Rendering



The Flask server logs show development mode traffic executing on port 8080 with HTTP 200 OK statuses. Subsequent cURL requests confirm the server successfully renders styled HTML structure containing header tags and linked stylesheets.

PART 5: Shell Script for Dockerfile and Build Automation

```
#!/bin/bash
mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static

cp sample-app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.

echo "FROM python" >> tempdir/Dockerfile
echo "RUN pip install --no-cache-dir --progress-bar off flask" >> tempdir/Dock
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample-app.py /home/myapp/" >> tempdir/Dockerfile
echo "EXPOSE 8080" >> tempdir/Dockerfile
echo "CMD python3 /home/myapp/sample-app.py" >> tempdir/Dockerfile

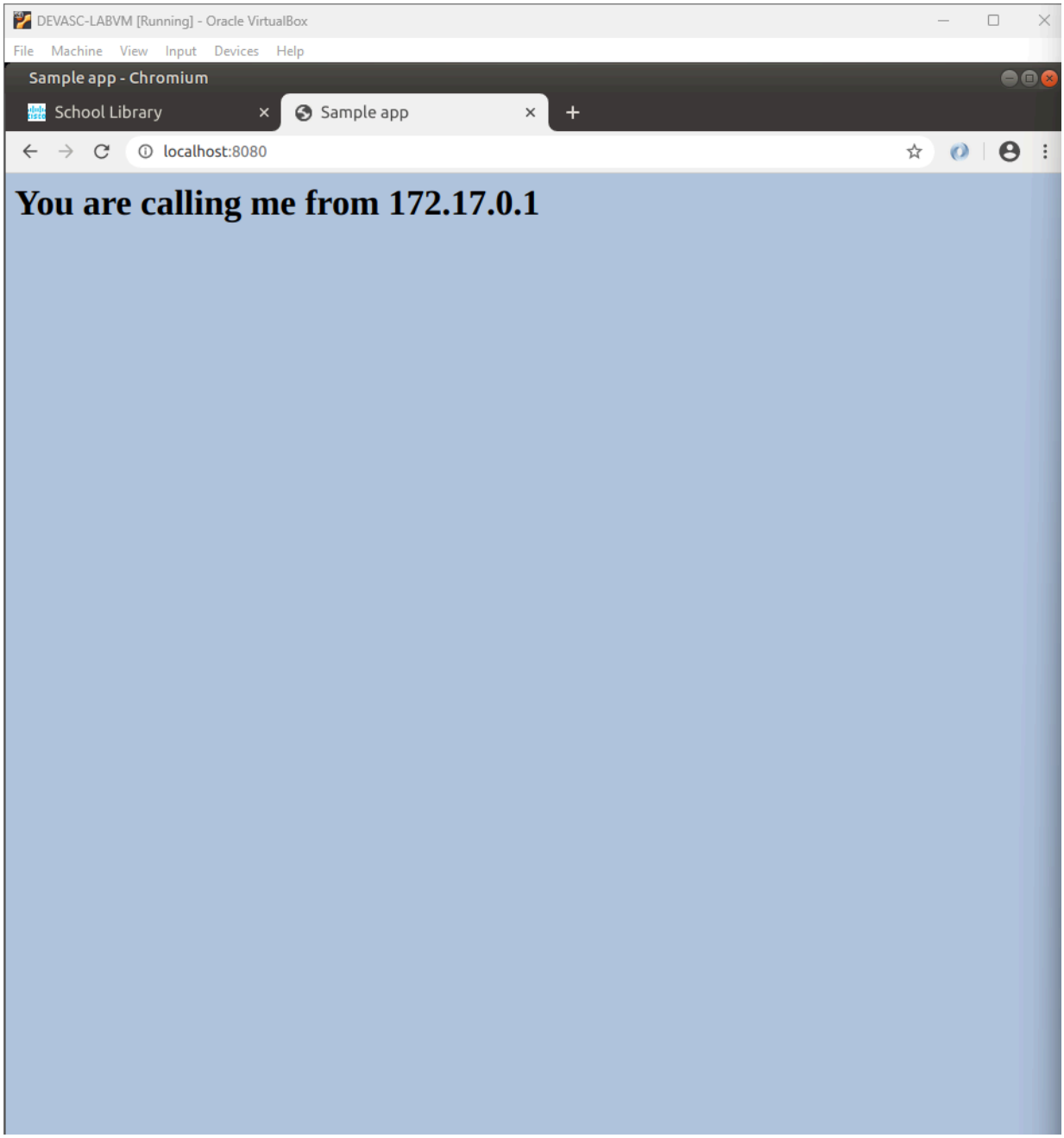
cd tempdir
docker build -t sampleapp .
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
[ Read 21 lines ]
```

The bash script automates environment setup by populating temporary directories and generating a custom Dockerfile. The resulting Dockerfile configures a Python base image, copies required app files, exposes port 8080, and triggers the container build process.

PART 6: Docker Container Build, Deployment, Browser Verification, and Cleanup

```
File  Edit  View  Search  Terminal  Help
 ---> b51535f89e4d
Successfully built b51535f89e4d
Successfully tagged sampleapp:latest
57dbb5d8f90a3fac6b0a6b6ec6212d02765bdcf32e841681dc8748d7049639c0
CONTAINER ID        IMAGE               COMMAND                  CREATED
     STATUS                      PORTS                     NAMES
57dbb5d8f90a        sampleapp           "/bin/sh -c 'python3…"   1 second ago
     Up Less than a second       0.0.0.0:8080->8080/tcp    samplerunning
98bdeba327d7        6f62299a884c        "/bin/sh -c 'pip ins…"   14 minutes ago
     Exited (2) 14 minutes ago                             distracted_ganguly
e9fb40c0bb8e        6f62299a884c        "/bin/sh -c 'pip ins…"   18 minutes ago
     Exited (2) 18 minutes ago                             sleepy_taussig
ed92426c9bb1        6f62299a884c        "/bin/sh -c 'pip ins…"   24 minutes ago
     Exited (2) 24 minutes ago                             naughty_grothendieck
devasc@labvm:~/labs/devnet-src/sample-app$ curl http://localhost:8080
<html>
<head>
    <title>Sample app</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>You are calling me from 172.17.0.1</h1>
</body>
</html>devasc@labvm:~/labs/devnet-src/sample-app$ ss
```

The Docker image was successfully built and launched as container *samplerunning* on port 8080. A cURL test verified web connectivity, identifying the bridge gateway IP *172.17.0.1*



Accessing *http://localhost:8080* via Chromium renders the web application interface properly. The browser displays the styled heading confirming caller IP address *172.17.0.1*.

```
</head>
<body>
    <h1>You are calling me from 172.17.0.1</h1>
</body>
</html>devasc@labvm:~/labs/devnet-src/sample-app$ docker exec -it samplerunning
shin/ba
root@57dbb5d8f90a:/# ls /home/myapp
sample-app.py  static  templates
root@57dbb5d8f90a:/# exit
exit
devasc@labvm:~/labs/devnet-src/sample-app$ docker stop samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker rm samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a
CONTAINER ID        IMAGE               COMMAND                  CREATED
     STATUS                      PORTS               NAMES
98bdeba327d7        6f62299a884c        "/bin/sh -c 'pip ins…"   21 minutes ago
     Exited (2) 21 minutes ago                       distracted_ganguly
e9fb40c0bb8e        6f62299a884c        "/bin/sh -c 'pip ins…"   25 minutes ago
     Exited (2) 24 minutes ago                       sleepy_taussig
ed92426c9bb1        6f62299a884c        "/bin/sh -c 'pip ins…"   31 minutes ago
     Exited (2) 31 minutes ago                       naughty_grothendieck
devasc@labvm:~/labs/devnet-src/sample-app$
```

An interactive shell session verified internal container app files under */home/myapp*. Stopping and removing the container successfully cleaned up active execution instances.

Conclusion

This activity provided comprehensive hands-on experience in configuring a Linux development environment, automating tasks using Bash scripting, and developing web applications with Python Flask. Additionally, it demonstrated the full workflow of containerizing applications using Docker—from writing build automation scripts and Dockerfiles to compiling images, deploying containers, verifying web interfaces, and managing container lifecycles effectively.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.